

Chapter 1

GUI Design & Event-Driven Programming (Windows Forms)

ENEL712 Embedded Systems Design — Lecture 21

Summary

Windows Forms GUIs in C#: the toolbox of common controls, containers, event-driven programming with handlers, error handling, dialogs, and multi-form encapsulation.

Topics Covered

- Introduction to the GUI
- Visual Studio toolbox & common controls
- Containers and non-visual controls
- Data handling and conversion
- Event-driven programming
- Error handling and dialogs
- Advanced topics (multiple forms, encapsulation)

1.1 Introduction to the GUI

- GUI is the primary user interaction surface.
- Visual Studio provides drag/drop/program controls.
- Custom controls (LEDs, 7-segment displays) can be downloaded or created.
- Example: a weather app combining labels, charts, calendars, text boxes, buttons, combo boxes, and picture boxes.

1.2 Visual Studio Toolbox & Common Controls

Control	Purpose
Label	Display read-only text; main property is Text.
Button	Triggers a Click event; key props: Name, Text, Enabled, Visible, Location, Size.
TextBox	User input or program output; events include TextChanged, Enter, Leave; supports multi-line.
ComboBox	Drop-down list; items via Collection editor or at run time.
NumericUpDown	Pick integer values within Minimum/Maximum/Value range.
CheckBox & RadioButton	Multi-select vs single-select among related options.

1.3 Containers and Non-Visual Controls

- Containers (GroupBox, Panel, TabControl) group related controls visually and manage shared properties (e.g., visibility).
- Non-visual controls (Timers, MenuStrips, FileDialogs) live in the designer tray below the form.

1.4 Data Handling and Conversion

- TextBoxes always handle text as strings.
- String → number: `Int32.Parse(string)`, `double.Parse(string)`, or `Convert.ToInt32(string)`.
- Number → string: `.ToString()`.

1.5 Event-Driven Programming

- Windows is event-driven — the program responds to clicks, key presses, etc.
- Event handlers are functions that respond to messages in the queue (auto-created by double-clicking a control).
- Critical: rename controls (e.g., `textBox1` → `openTBox`) before generating handlers; renaming after breaks the link.

1.6 Error Handling and Dialogs

- Wrap risky operations (parsing) in try/catch to avoid crashes.
- Calculator example: parse with try and handle failure with `catch(Exception ex)`.
- `MessageBox.Show(text, title, buttons)` shows information or errors — itself an OOP class.

1.7 Advanced: Multiple Forms & Encapsulation

- Apps can hold multiple forms — created via Solution Explorer or programmatically (manual control layout required).

- Form controls are typically declared private; expose access through public methods (e.g., `write_aTBox(string newText)`) to keep encapsulation.

1.8 Use Cases & Applications

- Companion PC apps for embedded products: configuration tools, firmware updaters, live telemetry viewers.
- Lab utilities: a small calculator, sweep generator, or scope-style logger built from a Form, a Chart, and a SerialPort.
- Quick prototypes: WinForms is still the fastest way to get a clickable interface in front of a customer.
- MAUI / WPF / Avalonia replace WinForms for new cross-platform GUIs but the event-driven mindset transfers directly.

1.9 Worked Example: Two-Number Calculator

```
private void plusB_Click(object sender, EventArgs e)
{
    try
    {
        double a = double.Parse(aTBox.Text);
        double b = double.Parse(bTBox.Text);
        resultLbl.Text = (a + b).ToString("F3");
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error parsing input: " + ex.Message,
            "Parsing Error",
            MessageBoxButtons.OK,
            MessageBoxIcon.Error);
    }
}
```

Notice the names `aTBox/bTBox/resultLbl` — set those before generating the Click handler. Renaming controls afterwards leaves the handler signature wired to the old name and Visual Studio cannot match them up.

1.10 Encapsulating Cross-Form Access

```
// In MainForm -- keep the control private and expose a deliberate API.
private System.Windows.Forms.TextBox statusTBox;

public void WriteStatus(string text)
{
    if (statusTBox.InvokeRequired)
        statusTBox.Invoke((Action)(() => statusTBox.Text = text));
    else
        statusTBox.Text = text;
}
```

Wrapping with `InvokeRequired` marshals back to the UI thread when calling from a worker thread (e.g., a `SerialPort.DataReceived` handler) — the most common WinForms threading bug.

1.11 Common Pitfalls

- Renaming a control after generating its handler — Visual Studio loses the link and the event silently stops firing.
- Calling MessageBox / UI updates from a non-UI thread — throws or hangs the form.
- Mutating the same TextBox.Text repeatedly during a tight loop redraws on every change; batch updates with SuspendLayout/ResumeLayout if perf matters.
- Storing all logic in event handlers — extract real work into separate methods or classes so it's testable.

Key Definitions

GUI

Graphical User Interface — visual layer for interacting with a program.

Event-Driven Programming

Computer responds to user/system events such as clicks and key presses.

Event Handler

Function executed in response to a specific event (e.g., button Click).

Encapsulation

Hiding form internals (private) and exposing controlled access through public methods.

Control Naming

Naming controls before creating handlers to keep code/handler links stable.

Try/Catch

Error-handling structure wrapping risky code to prevent crashes.

Container

Control grouping related GUI elements (GroupBox, TabControl).

Dialog

Window class such as MessageBox used to display info or prompt the user.

Event-Driven Programming

Programs whose flow is driven by external events (user input, messages) routed to handler functions.

UI Thread

Single thread that owns the message loop; almost all WinForms control accesses must happen on this thread.

Message Loop

Loop that pulls Windows messages from the queue and dispatches them to control handlers.

Modal Dialog

Dialog that blocks input to the rest of the application until it is dismissed (e.g., MessageBox.Show).

Sources

- Lecture 21.pdf